Estatísticas de Ordem

i-ésima estatística de ordem

A *i*-ésima estatística de ordem de um conjunto de *n* elementos é o *i*-ésimo menor elemento.



Problema da Seleção

Dados:
Lista L com n elementos (distintos) e inteiro i, $1 \leq i \leq n$

Problema:
Retornar o elemento x de L que é maior que exatamente i-1 elementos de L.

Mediana



1º menor elemento
=
Mínimo

*n*º menor elemento
=
Máximo

Ponto médio
=
Mediana

Mediana

n ímpar: $i = (n+1)/2$ (mediana)

n par: $i = n/2$ (mediana inferior)

$i = n/2 + 1$ (mediana superior)

obs: Estaremos nos referindo à mediana inferior

Problema da Seleção

Solução 1: Ordena *L* e retorna a *i*-ésima estatística de ordem



Complexidade: Ordenação – $O(n \log n)$

Selecionar Mínimo e Máximo

Algoritmos simples

Mínimo (máximo)
n -1 comparações

Mínimo e máximo simultâneos
2(n-2) comparações

Mínimo e Máximo Simultâneos

São necessárias $3\lfloor n/2 \rfloor$ comparações

Exercício:

Fazer o algoritmo que encontre o mínimo e o máximo de uma lista com n elementos utilizando $3\lfloor n/2 \rfloor$ comparações.



Seleção em Tempo Linear

- Complexidade média linear
- Complexidade linear em pior caso

Seleção em Tempo Linear Esperado

Técnica: Dividir e Conquistar

Algoritmo modificado a partir do Quicksort.

Quicksort: processa os dois lados da partição

Quickselect: processa um lado após partição

Seleção em Tempo Linear Esperado

Pivô: selecionado aleatoriamente com equiprobabilidade.

Particiona(*L*,*p*,*r*)

i ← random(*p*,*r*); troca(*p*,*i*)
v ← *L*[*p*]; *i* ← *p*+1; *j* ← *r*
Enquanto *i* ≤ *j*
    enquanto *L*[*i*] < *v* faça *i* ← *i* + 1
    enquanto *L*[*j*] ≥ *v* faça *j* ← *j* - 1
    se *i* < *j* então Troca(*L*[*i*],*L*[*j*])
Fim_enquanto
L[*p*] ← *L*[*j*] ; *L*[*j*] ←*v*
retorna(*j*,*L*)

Seleção em Tempo Linear Esperado

```
Quickselect(L,p,r,i)
  se (p = r) retorna(L[p])
  q ← Particiona(L,p,r)
   k ← q – p + 1
   se (i = k) retorna(L[q])            **elemento encontrado**
   se (i < k) Quickselect(L,p,q-1,i)
   senão Quickselect(L,q+1,r,i-k)
```

Seleção em Tempo Linear Esperado

Exemplo: $i = 3$



Quickselect(*L,p,r,i*)

se ($p = r$) retorna($L[p]$)
$q \leftarrow$ Particiona(L,p,r)
$k \leftarrow q - p + 1$
se ($i = k$) retorna($L[q]$) **
se ($i < k$) Quickselect(L,p,q-1,i)
senão Quickselect(L,q+1,r,i-k)

Seleção em Tempo Linear Esperado

p=1 *r*=11

15	12	1	5	45	81	78	2	49	30	23

Supor *i* = 7

Troca elementos das posições 1 e 7

v = 78

78	12	1	5	45	81	15	2	49	30	23
					i					*j*

Troca elementos das posições *i* e *j* e repete

78	12	1	5	45	23	15	2	49	30	81
									j	*i*

L[1] ← 30 *L*[*j*] ← *v*

30	12	1	5	45	23	15	2	49	78	81

Particiona(*L*,*p*,*r*)

i ← random(*p*,*r*); troca(*p*,*i*)
v ← *L*[*p*]; *i* ← *p*+1; *j* ← *r*
Enquanto *i* ≤ *j*
    enquanto *L*[*i*] < *v* faça *i* ← *i* + 1
    enquanto *L*[*j*] ≥ *v* faça *j* ← *j* - 1
    se *i* < *j* então Troca(*L*[*i*],*L*[*j*])
Fim_enquanto
L[*p*] ← *L*[*j*] ; *L*[*j*] ←*v*
retorna(*j*,*L*)

Seleção em Tempo Linear Esperado

Exemplo: $i = 3$



Quickselect(*L,p,r,i*)

```
se (p = r) retorna(L[p])
q ← Particiona(L,p,r)
k ← q – p + 1
se (i = k) retorna(L[q])
se (i < k) Quickselect(L,p,q-1,i)
senão Quickselect(L,q+1,r,i-k)
```

$k = 10 – 1 + 1 = 10$

**

Seleção em Tempo Linear Esperado

p=1 *r*=9

30	12	1	5	45	23	15	2	49	78	81

Supor *i* = 2

Troca elementos das posições 1 e 2

v = 12

12	30	1	5	45	23	15	2	49	78	81

i *j*

12	2	1	5	45	23	15	30	49	78	81

j *i*

$L[1] \leftarrow 5$ $L[j] \leftarrow v$

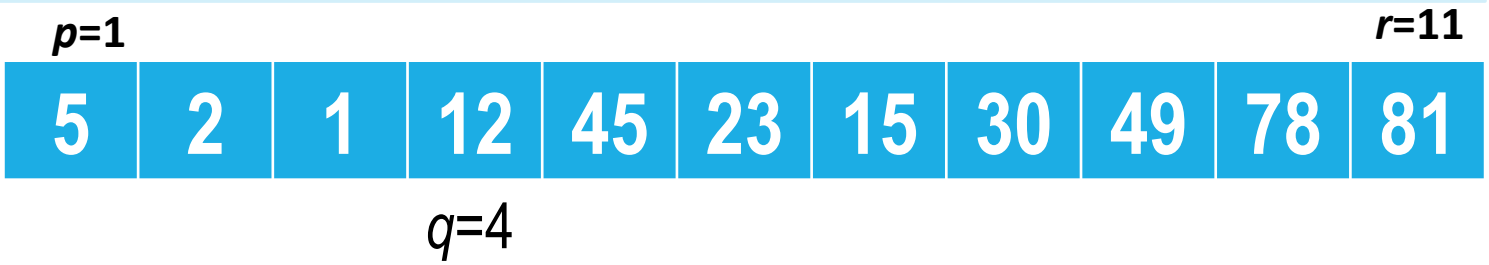
5	2	1	12	45	23	15	30	49	78	81

Particiona(*L*,*p*,*r*)

$i \leftarrow$ random(p,r); troca(p,i)
$v \leftarrow L[p]$; $i \leftarrow p+1$; $j \leftarrow r$
Enquanto $i \leq j$
    enquanto $L[i] < v$ faça $i \leftarrow i + 1$
    enquanto $L[j] \geq v$ faça $j \leftarrow j - 1$
    se $i < j$ então Troca($L[i]$,$L[j]$)
Fim_enquanto
$L[p] \leftarrow L[j]$; $L[j] \leftarrow v$
retorna(j,L)

Seleção em Tempo Linear Esperado

Exemplo: $i = 3$



Quickselect(*L,p,r,i*)

se ($p = r$) retorna($L[p]$)

$q \leftarrow$ Particiona(L,p,r)

$k \leftarrow q - p + 1$ ← $k = 4$

se ($i = k$) retorna($L[q]$) **

se ($i < k$) Quickselect($L,p,q-1,i$) ←

senão Quickselect($L,q+1,r,i-k$)

Seleção em Tempo Linear Esperado

p=1 *r*=3

5	2	1	12	45	23	15	30	49	78	81

Supor *i* = 3

Troca elementos das posições 1 e 3

v = 1

1	2	5	12	45	23	15	30	49	78	81

j *i*

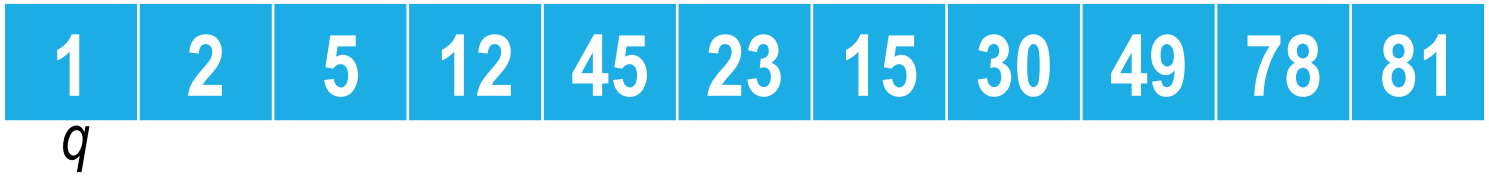
$L[1] \leftarrow 1$ $L[j] \leftarrow v$

Particiona(*L*,*p*,*r*)

i ← random(*p*,*r*); troca(*p*,*i*)
v ← *L*[*p*]; *i* ← *p*+1; *j* ← *r*
Enquanto $i \leq j$
    enquanto $L[i] < v$ faça $i \leftarrow i + 1$
    enquanto $L[j] \geq v$ faça $j \leftarrow j - 1$
    se $i < j$ então Troca(*L*[*i*],*L*[*j*])
Fim_enquanto
$L[p] \leftarrow L[j]$; $L[j] \leftarrow v$
retorna(*j*,*L*)

Seleção em Tempo Linear Esperado

Exemplo: $i = 3$



Quickselect(*L,p,r,i*)

```
se (p = r) retorna(L[p])
q ← Particiona(L,p,r)
k ← q – p + 1
se (i = k) retorna(L[q])
se (i < k) Quickselect(L,p,q-1,i)
senão Quickselect(L,q+1,r,i-k)
```

**

k = 1

p = 2
r =3
i-k = 2

Seleção em Tempo Linear Esperado

p=2 *r*=3

1	2	5	12	45	23	15	30	49	78	81

Supor $i = 2$

Troca elementos das posições 2 e 2

$v = 2$

1	2	5	12	45	23	15	30	49	78	81

j *i*

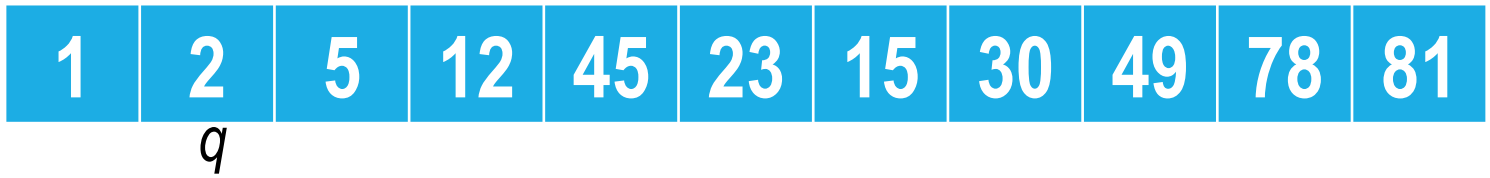
$L[2] \leftarrow 2$ $L[j] \leftarrow v$

Particiona(*L*,*p*,*r*)

```
i ← random(p,r); troca(p,i)
v ← L[p]; i ← p+1; j ← r
Enquanto i ≤ j
    enquanto L[i] < v faça i ← i + 1
    enquanto L[j] ≥ v faça j ← j - 1
    se i < j então Troca(L[i],L[j])
Fim_enquanto
L[p] ← L[j] ; L[j] ←v
retorna(j,L)
```

Seleção em Tempo Linear Esperado

Exemplo: $i = 2$



Quickselect(*L*,*p*,*r*,*i*)

se ($p = r$) retorna($L[p]$)

$q \leftarrow$ Particiona(L,p,r)

$k \leftarrow q - p + 1$ ← $k = 1$

**

se ($i = k$) retorna($L[q]$)

se ($i < k$) Quickselect(L,p,q-1,i)

senão Quickselect(L,q+1,r,i-k) ← $p = 3$, $r = 3$, $i-k = 1$

Seleção em Tempo Linear Esperado

p=3
r=3

1	2	5	12	45	23	15	30	49	78	81

i = 3

Troca elementos das posições 3 e 3

v = 5

1	2	5	12	45	23	15	30	49	78	81

j *i*

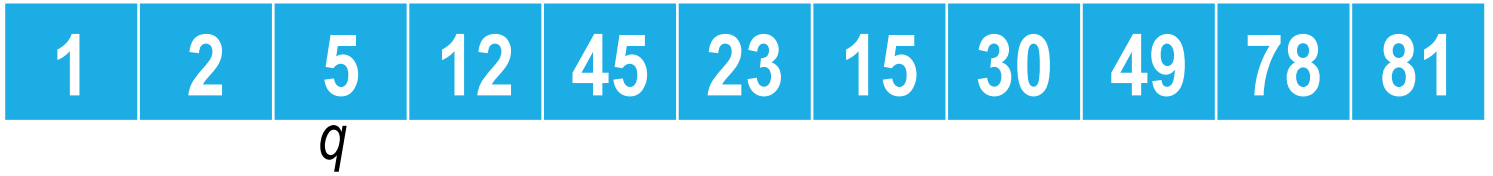
L[3] ← 5 *L*[*j*] ← *v*

Particiona(*L*,*p*,*r*)

i ← random(*p*,*r*); troca(*p*,*i*)
v ← *L*[*p*]; *i* ← *p*+1; *j* ← *r*
Enquanto *i* ≤ *j*
    enquanto *L*[*i*] < *v* faça *i* ← *i* + 1
    enquanto *L*[*j*] ≥ *v* faça *j* ← *j* - 1
    se *i* < *j* então Troca(*L*[*i*],*L*[*j*])
Fim_enquanto
L[*p*] ← *L*[*j*] ; *L*[*j*] ←*v*
retorna(*j*,*L*)

Seleção em Tempo Linear Esperado

Exemplo: $i = 1$



Quickselect(*L,p,r,i*)

```
se (p = r) retorna(L[p])
q ← Particiona(L,p,r)
k ← q – p + 1
se (i = k) retorna(L[q])
se (i < k) Quickselect(L,p,q-1,i)
senão Quickselect(L,q+1,r,i-k)
```

k = 1

** Fim

Seleção em Tempo Linear Esperado

Análise do Quickselect

Qual o tempo de pior caso para o Quickselect?

```
Quickselect(L,p,r,i)
  se (p = r) retorna(L[p])
  q ← Particiona(L,p,r)
   k ← q – p + 1
   se (i = k) retorna(L[q])
   se (i < k) Quickselect(L,p,q-1,i)
   senão Quickselect(L,q+1,r,i-k)
```

Seleção em Tempo Linear Esperado

Pior Caso $O(n^2)$



Seleção em Tempo Linear Esperado

Tempo Esperado

$T(n)$ é uma variável aleatória com valor esperado E[$T(n)$].

Considera-se equiprobabilidade na escolha do pivô.

Variável aleatória $X_k \in \{0,1\}$, $k = 1,...,n$.
$X_k = 1$ (o elemento na posição k foi escolhido como pivô)
$X_z = 0$, $z \neq k$, $1 \leq z \leq n$.

E[X_k] = $1/n$ (equiprobabilidade)

Seleção em Tempo Linear Esperado

Tempo Esperado

Quando $X_k = 1$, as listas sobre as quais a recursão pode ser feita têm tamanhos k-1 e n-k, sendo: $L[p...q\text{-}1]$ e $L[q+1...r]$.

Supor que o i-ésimo elemento esteja sempre na (sub)lista de maior tamanho nas chamadas recursivas, assim

$$T(n) \leq \sum_{k=1}^{n} X_k \left(T(\max(k-1, n-k)) + O(n)\right)$$

Chamada recursiva

Partição

Seleção em Tempo Linear Esperado

Tempo Esperado

Para valores esperados, teremos:

$$E[T(n)] \leq E\left[\sum_{k=1}^{n} X_k \left(T(\max(k-1, n-k)) + O(n)\right)\right]$$

Pela linearidade

$$E[T(n)] \leq \sum_{k=1}^{n} E[X_k T(\max(k-1, n-k))] + O(n)$$

Seleção em Tempo Linear Esperado

Tempo Esperado

Quando *n* variáveis aleatórias são independentes:

$$E[X_1 \ldots, X_n] \leq E[X_1]E[X2]\ldots E[X_n]$$

Exercício: Provar que X_k e $T(\max(k\text{-}1, n\text{-}k))$ são variáveis aleatórias independentes.



Seleção em Tempo Linear Esperado

Tempo Esperado

$$E[T(n)] \leq \sum_{k=1}^{n} E[X_k] E[T(\max(k-1, n-k))] + O(n)$$

Sabemos que : $E[X_k] = \frac{1}{n}$

$$E[T(n)] \leq \sum_{k=1}^{n} \frac{1}{n} E[T(\max(k-1, n-k))] + O(n)$$

Seleção em Tempo Linear Esperado

Tempo Esperado

Em relação ao termo $T(\max(k\text{-}1, n\text{-}k))$ no somatório, temos que:

se n é par, cada termo de $T(\lceil n/2 \rceil)$ até $T(n\text{-}1)$ aparece 2 vezes no somatório

se n é ímpar, cada termo de $T(\lceil n/2 \rceil)$ até $T(n\text{-}1)$ aparece 2 vezes no somatório e o termo $T(\lfloor n/2 \rfloor)$ aparece 1 vez.

$k=4$ $n=10$
$k=7$ $n=10$

Seleção em Tempo Linear Esperado

Tempo Esperado

$$E[T(n)] \leq \frac{2}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} E[T(k)] + O(n)$$

Resolver a recorrência por substituição com as seguintes suposições:

- $T(n) \leq cn$ para alguma constante c que satisfaça as condições iniciais da recorrência.
- $T(n)$ é $O(1)$ para $n <$ "alguma constante" (esta constante será calculada adiante)
- O termo $O(n)$ é limitado superiormente por an, para todo $n > 0$ e a constante

Seleção em Tempo Linear Esperado

Tempo Esperado

$$E[T(n)] \leq \frac{2}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} ck + an$$

Resolvendo a recorrência:
(exercício – resolver a inequação)

$$E[T(n)] \leq cn - \left(\frac{cn}{4} - \frac{c}{2} - an\right)$$

Mostrar que para *n* suficientemente grande, esta expressão é, no máximo, *cn*.

Seleção em Tempo Linear Esperado

Tempo Esperado

Escolher c tal que: $\frac{cn}{4} - \frac{c}{2} - an < 0$

$$n < \frac{2c}{c - 4a}$$

Se considerarmos que $T(n)$ é $O(1)$ para $n < 2c/(c-4a)$, então $T(n)$ é $O(n)$.

Seleção em Tempo Linear no Pior Caso



Seleção em Tempo Linear no Pior Caso

Como prevenir que os piores casos aconteçam no algoritmo anterior?

Selecionar o pivô com cuidado para que não ocorram subproblemas “muito” grandes.

Seleção em Tempo Linear no Pior Caso

Uma boa escolha para o pivô é a **mediana**.

Se for possível escolher sempre a mediana, então os subproblemas terão tamanho no máximo $n/2$.

Isto resulta na recorrência:

$$T(n) = T(n/2) + O(n)$$

que é $O(n)$.

Exercício: Resolver a recorrência



Seleção em Tempo Linear no Pior Caso

Como garantir que o pivô é a **mediana**?



Seleção em Tempo Linear no Pior Caso

Como garantir que o pivô é a **mediana**?

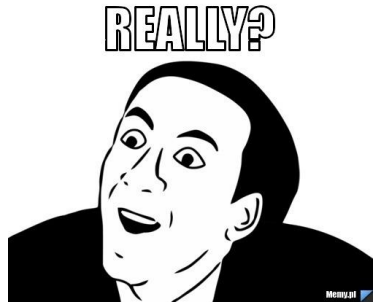
Não vamos encontrar a **mediana** exata, apenas garantir que o tamanho dos dois subproblemas não seja maior que uma fração fixa do problema principal.

Isto resulta do fato de que:
$T(n) = T(an) + cn$ é $O(n)$ para constante a , $0 < a < 1$.

Seleção em Tempo Linear no Pior Caso

Como garantir que o pivô é a **mediana**?

A aproximação da **mediana** é feita por amostragem.



Seleção em Tempo Linear no Pior Caso

Algoritmo *Select*

1. Se $n < 50$ então ordene os elementos de L e retorne o i-ésimo elemento.

2. Dividir os n elementos de L em $\lfloor n/5 \rfloor$ grupos de 5 elementos cada e, no máximo, um grupo com os n mod 5 elementos restantes.

3. Encontrar a mediana de cada um dos $\lceil n/5 \rceil$ grupos
   (usar um algoritmo de ordenação e pegar a mediana)

4. Usar *Select* recursivamente para encontrar a mediana x das $\lceil n/5 \rceil$ medianas (se o número for par, então toma-se a mediana inferior).

5. Particionar L em torno de x (usando o algoritmo determinístico de partição do quicksort).

Seleção em Tempo Linear no Pior Caso

Algoritmo *Select*

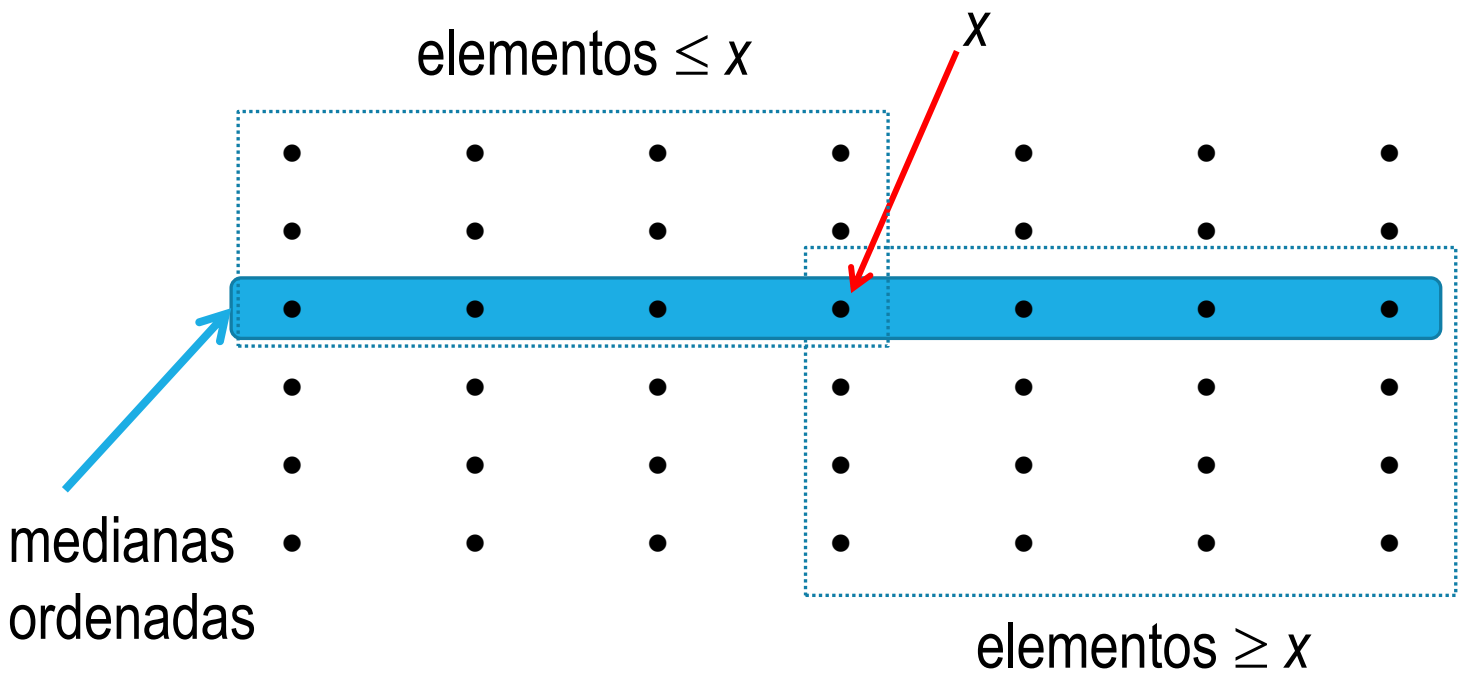
6. Seja *k* uma unidade maior que o número de elementos no lado baixo da partição, de forma que *x* é o *k*-ésimo menor elemento e existam *n*-*k* elementos no lado alto da partição.

Se *i* = *k* retorne *x*.
Se *i* < *k*
    usar *Select* recursivamente para encontrar o *i-ésimo* elemento no lado baixo da partição
senão
    usar *Select* recursivamente para encontrar o (*i*-*k*)-*ésimo* elemento no lado alto da partição

Seleção em Tempo Linear no Pior Caso

Análise do Algoritmo *Select*

Determinar um limite inferior sobre o número de elementos que são maiores que o elemento x.



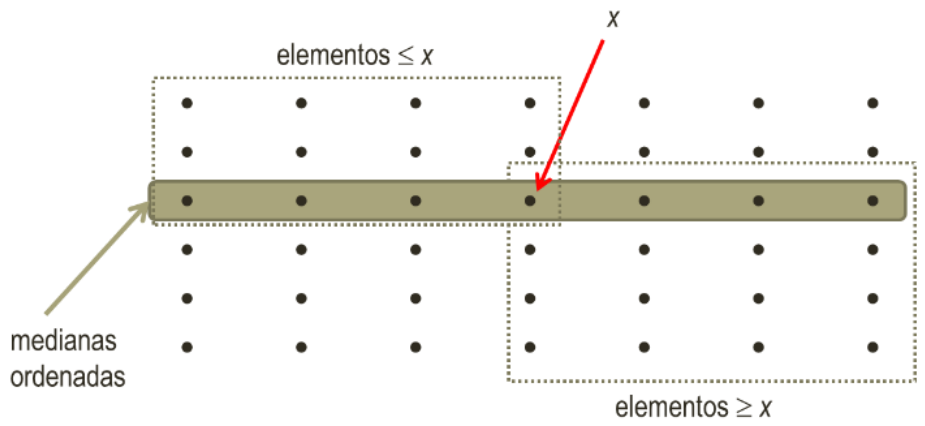
Seleção em Tempo Linear no Pior Caso

Análise do Algoritmo *Select*

Pelo menos metade das medianas encontradas é maior que x.
Logo, metade dos $\lceil n/5 \rceil$ grupos contribui com 3 elementos maiores que x, exceto pelo único grupo com menos de 5 elementos (se n não dividir 5 exatamente) e pelo único grupo contendo o próprio x.

Descontando estes dois grupos temos que o número de elementos maiores que x é pelo menos:

$$3\left(\left\lceil \frac{1}{2}\left\lceil \frac{n}{5} \right\rceil \right\rceil - 2\right) \geq \frac{3n}{10} - 6$$



Seleção em Tempo Linear no Pior Caso

Análise do Algoritmo *Select*

Pelo mesmo raciocínio temos que o número de elementos menores que x é no mínimo $\frac{3n}{10} - 6$

Logo, no pior caso, temos no máximo $n - \left(\frac{3n}{10} - 6\right) = \frac{7n}{10} + 6$ elementos maiores que x.

Portanto, o *Select* na etapa 6 tem pior caso $T(7n/10 + 6)$.

Seleção em Tempo Linear no Pior Caso

Análise do Algoritmo *Select*

As etapas 2, 3 e 5 demoram $O(n)$, uma vez que a etapa 3 tem $O(n)$ chamadas para conjuntos de tamanho $O(1)$.

A etapa 4 demora $T(\lceil n/5 \rceil)$.

Cálculo de $T(n)$:

$$T(n) \leq \begin{cases} \Theta(1) & \text{se } n \leq \text{Constante} \\ T(\lceil n/5 \rceil) + T(7n/10 + 6) + O(n) & \text{se } n > \text{Constante} \end{cases}$$

Seleção em Tempo Linear no Pior Caso

Análise do Algoritmo *Select*

Cálculo de $T(n)$:

$$T(n) \leq \begin{cases} \Theta(1) & \text{se } n \leq 140 \\ T(\lceil n/5 \rceil) + T(7n/10 + 6) + O(n) & \text{se } n > 140 \end{cases}$$

Valor 140 será visto adiante.

Seleção em Tempo Linear no Pior Caso

Análise do Algoritmo *Select*

Mostra-se que $T(n) \leq cn$ para alguma constante c grande o bastante para todo $n > 0$.

Começa mostrando que $T(n) \leq cn$ para alguma constante c grande o bastante para todo $n \leq 140$.

Escolhe-se uma constante a tal que a parcela $O(n)$ seja limitada superiormente por an para todo $n > 0$.

Seleção em Tempo Linear no Pior Caso

Análise do Algoritmo *Select*

Temos:

$$T(n) \leq c\lceil n/5 \rceil + c(7n/10 + 6) + an$$

$$T(n) \leq 9cn/10 + 7c + an$$

$$T(n) \leq cn + (-cn/10 + 7c + an)$$

Para aparecer o termo *cn*

$T(n)$ é menor que cn se

$$-cn/10 + 7c + an \leq 0$$

Seleção em Tempo Linear no Pior Caso

Análise do Algoritmo *Select*

A desigualdade $-cn/10 + 7c + an \leq 0$

é equivalente a $c \geq 10a(n/(n-70))$

quando $n > 70$.

Considerando que $n \geq 140$, temos $n/(n-70) \leq 2$

Portanto, a escolha de $c \geq 20a$ satisfará a desigualdade.

Obs: qualquer constante maior que 70 poderia ser escolhida.